Day 1 — RAG Foundations and Bedrock Knowledge Bases

Tuesday 13 Oct 2026 · Target exams: AWS Certified Generative AI Developer – Professional (Sat 7 Nov) and Microsoft AI-103 (Sun 8 Nov) · Focus: AWS first

Today in one paragraph
RAG (Retrieval-Augmented Generation) lets an LLM answer from your data without retraining it: documents are prepared offline (ingest, parse, chunk, embed, index) and, for every question, the most relevant chunks are retrieved and pasted into the prompt so the model answers from them and can cite them. On AWS the managed way to do all of this is **Amazon Bedrock Knowledge Bases**. In the lab you built one on S3, tested default vs hierarchical chunking and semantic vs hybrid search.

1. What RAG is and why it exists

- **Problem:** a foundation model only knows its training data — nothing private, nothing recent — and it will confidently invent answers (hallucinate) when it doesn't know.
- **Idea:** search your own knowledge first, then give the model the relevant passages and instruct it to answer only from them. Think of it as a search engine whose top results are pasted into a prompt template.
- **Benefits:** fresh data without retraining, citations / traceability, fewer hallucinations, access control at retrieval time, much cheaper than fine-tuning.
- **RAG vs fine-tuning:** knowledge that changes often → RAG. Changing the model's style, tone, format or domain behaviour → fine-tuning. Need both → do both.

2. The pipeline

```
OFFLINE  (indexing — runs on a schedule / when data changes)
  1 Ingest -> 2 Parse -> 3 Chunk -> 4 Embed -> 5 Index/Store

ONLINE   (every user question)
  6 Query rewrite -> 7 Embed query -> 8 Retrieve -> 9 Rerank
     -> 10 Augment prompt -> 11 Generate -> 12 Cite & check grounding
```

Exam mindset: every RAG scenario question is really asking **which step is broken** and **which managed feature fixes it**.

3. Each step — what it is, AWS tool, Azure tool

Step	What it does	AWS	Azure
1. Ingest	Pull source data in from where it lives; re-run when it changes.	S3 (main KB source); connectors: web crawler, Confluence, SharePoint, Salesforce; custom/direct ingestion API; **sync (ingestion) jobs**	Blob Storage / ADLS, Cosmos DB, Azure SQL, SharePoint via **AI Search indexers** (scheduled)
2. Parse	Turn PDFs, Word, scans, tables into clean text + structure. Bad parsing poisons everything after.	KB default parser; **foundation-model parsing** or **Bedrock Data Automation** for complex PDFs/tables/images; Textract (OCR)	**Document Intelligence** (Layout model); AI Search document cracking
3. Chunk	Split documents into passages. Too big = noise; too small = lost meaning.	Default (~300 tokens), fixed-size, **hierarchical**, **semantic**, none, or custom via **Lambda**	Text Split skill (pages/sentences); Document Layout skill (structure-aware)

Step	What it does	AWS	Azure
4. Embed	Convert each chunk into a vector capturing its meaning; similar meaning = nearby vectors.	**Titan Text Embeddings V2** (256/512/1024 dims), Titan Multimodal Embeddings, Cohere Embed	Azure OpenAI **text-embedding-3-small/large**; integrated vectorization in AI Search
5. Index / Store	Save vectors + text + metadata in a store built for similarity search.	**OpenSearch Serverless** (quick-create default), Aurora PostgreSQL (pgvector), **Neptune Analytics** (GraphRAG), S3 Vectors, Pinecone, MongoDB Atlas, Redis	**AI Search** vector index; Cosmos DB vector search
6. Query rewrite	Clarify or split a complex question into sub-queries before searching.	KB **query decomposition**	AI Search query rewriting / agentic retrieval
7. Embed query	Vectorise the question with the **same** embedding model used in step 4.	Same model as step 4 (automatic in KB)	Same model as step 4
8. Retrieve	Fetch top-K chunks: semantic (vector), keyword (BM25) or **hybrid**; apply metadata filters.	KB **Retrieve** API; semantic or hybrid; metadata filters	AI Search hybrid query (fused with **RRF**), filters, security trimming
9. Rerank	Second, smarter model re-scores results so the best chunk is first.	Bedrock **rerank models** (Amazon Rerank, Cohere Rerank)	AI Search **semantic ranker**
10. Augment	Build the prompt: instructions + retrieved chunks + question.	**RetrieveAndGenerate** prompt template; **Prompt Management**	Foundry prompt flow / templates; Semantic Kernel; Agent Service
11. Generate	LLM writes the answer from the augmented prompt.	Bedrock models (Claude, Nova, Llama, Mistral…) via **Converse / InvokeModel**	Azure OpenAI GPT models / Foundry deployments
12. Cite & check	Attach sources; verify the answer is supported by retrieved text.	RetrieveAndGenerate returns **citations**; **Guardrails contextual grounding check**	Citations from AI Search / Agent Service; **Content Safety groundedness detection**

Managed shortcuts (the 'least operational overhead' answers)

Need	AWS	Azure
Whole RAG pipeline managed	**Bedrock Knowledge Bases**	**AI Search** (integrated vectorization) + **Foundry Agent Service**
RAG as one tool of an agent	Bedrock Agents with a Knowledge Base attached	Foundry Agent Service with the AI Search / file search tool

4. Chunking strategies (Bedrock Knowledge Bases)

Strategy	How it works	Use when
Default	~300-token chunks, respects sentence boundaries.	Quick start; general text.
Fixed-size	You set max tokens per chunk + overlap %.	Uniform docs; you want predictable chunk size.
Hierarchical	Two layers: small **child** chunks are searched (precision); their larger **parent** chunk is returned to the model (context).	Long, structured docs (contracts, manuals) where an answer needs surrounding context.
Semantic	Splits where the meaning changes (uses an embedding model; costs more at ingestion).	Docs without clear structure where topics shift mid-section.

Strategy	How it works	Use when
None	Each file is one chunk.	Docs already pre-split (e.g. one FAQ per file).
Custom (Lambda)	Your own Lambda transforms/chunks content during ingestion.	Special formats or business rules none of the above handle.

5. Retrieval concepts

- **Semantic (vector) search:** matches by meaning — 'car broke down' finds 'vehicle failure'. Weak on exact tokens.
- **Keyword (BM25) search:** matches exact words — great for product codes, error IDs, names.
- **Hybrid search:** runs both and merges results. Fix for 'exact part numbers aren't found'.
- **Reranking:** right documents retrieved but in the wrong order → add a reranker.
- **Metadata filtering:** restrict results by attributes (department, date, tenant). Enforce access control here, never by trusting the prompt.
- **Retrieve** API = returns chunks only (you build the prompt / custom logic). **RetrieveAndGenerate** = full managed answer with citations.

6. Exam decision rules (memorise)

1. '**Least operational overhead**' → managed service (Knowledge Bases), not a custom Lambda pipeline.
2. Exact terms (codes, IDs, names) missed → **hybrid search**.
3. Good docs retrieved, wrong order → **reranker**.
4. Long docs lose surrounding context → **hierarchical chunking**.
5. Tables / complex PDFs parsed badly → **foundation-model parsing** or **Bedrock Data Automation**.
6. Users must only see permitted docs → **metadata filtering** at retrieval time.
7. Knowledge changes frequently → RAG + **scheduled sync**; style/format change → fine-tuning.
8. Relationships across connected entities → **GraphRAG** (Neptune Analytics).
9. Need custom logic between retrieval and generation → **Retrieve**; otherwise **RetrieveAndGenerate**.
10. Query and documents must use the **same embedding model** — a mismatch means nothing matches.

7. Evaluating RAG

Metric	Question it answers	Checks
Context precision	Are the retrieved chunks relevant?	Retrieval
Context recall	Did we retrieve everything needed?	Retrieval
Faithfulness / groundedness	Is the answer supported by the retrieved context?	Generation
Answer relevance	Does the answer actually address the question?	Generation

Tools: **Amazon Bedrock Evaluations** (RAG evaluation, LLM-as-a-judge) · Azure: **Foundry evaluations** (groundedness, relevance, coherence evaluators).

8. Lab work — what I did

1. Uploaded 5 documents to an **S3 bucket**.
2. Created a **Bedrock Knowledge Base** with a quick-create **OpenSearch Serverless** vector store.

3. Created a **data source** pointing at the bucket (default chunking) and **synced** it (ingestion job).
4. Tested in the console with default settings and evaluated the model's responses.
5. Added a **second data source** on the same documents with **hierarchical chunking**, and synced it.
6. Switched the search type to **hybrid** and increased the **number of retrieved results**.

9. Mistakes and corrections

What happened	Correction / learning
Described the setting as 'max chunking size for the search'.	Two different settings. **Chunk size is an ingestion-time setting** on the data source (before sync). At query time you set **number of results**, search type and filters. The exam mixes these up on purpose.
Needed a new data source to try hierarchical chunking.	Correct move: a data source's **chunking strategy can't be changed after creation**. Re-chunking = new data source (or new KB) + re-sync.
Both data sources point at the same 5 docs in one KB.	The KB now holds **duplicate content chunked two ways**, so results mix both — not a clean A/B test. Use separate KBs (or delete the old data source) when comparing strategies.
(Possible surprise) fewer results than requested with hierarchical chunking.	Expected: several child chunks can share one parent, and duplicate parents are merged. Not a bug.
Cost risk: OpenSearch Serverless left running after the lab.	It bills **per hour even when idle**. Delete the collection + KB after each lab; set a billing alarm.

10. Key takeaways

- RAG = offline indexing + online retrieve/augment/generate. Know all 12 steps and the AWS service for each.
- Bedrock Knowledge Bases is the managed answer; know its chunking options, vector-store options and the Retrieve vs RetrieveAndGenerate split.
- Chunking is decided at ingestion and is fixed per data source.
- Hybrid for exact terms, reranker for ordering, hierarchical for context, metadata filters for access control.
- Groundedness checks (Guardrails contextual grounding) are the defence against hallucinated answers.
- Clean experiments: change one variable at a time, in isolation.

11. Revision questions

Warm-up (answer from memory)

1. Name the 5 offline steps of RAG, in order.
2. Which API returns only chunks, and which returns a full answer with citations?
3. When does hybrid search beat pure semantic search? Give an example.
4. What does hierarchical chunking search over, and what does it return to the model?
5. Your docs change every night. How does the Knowledge Base stay current?

Scenario quiz

1. A support bot on Bedrock KB misses answers when users type exact part numbers like 'XJ-4471-B'; semantic questions work. Least-effort fix? (A) fine-tune the embedding model (B) hybrid search (C) bigger chunks (D) Lambda custom chunker
2. 80-page contracts: answers cite the right clause but miss definitions it depends on. Which chunking? (A) fixed 200-token (B) none (C) hierarchical (D) semantic with 0% overlap
3. An Azure app must refuse to answer when retrieved docs don't support the claim. Which feature?
4. Product catalogue changes nightly; keep the Bedrock KB current with no servers to manage. How?
5. Top-5 chunks are all somewhat relevant but the best one is usually ranked 4th or 5th. Which step, which fix?

Answers

#	Answer
W1	Ingest → Parse → Chunk → Embed → Index/Store.
W2	**Retrieve** = chunks only; **RetrieveAndGenerate** = generated answer + citations.
W3	When queries contain exact tokens — product codes, error IDs, names (e.g. 'XJ-4471-B').
W4	Searches small **child** chunks for precision; returns the larger **parent** chunk for context.
W5	A scheduled **ingestion (sync) job** — e.g. EventBridge Scheduler calling StartIngestionJob, or an S3-event-triggered sync.
Q1	**B** — hybrid search adds keyword matching for exact tokens.
Q2	**C** — hierarchical: precise child match, parent chunk carries the surrounding definitions.
Q3	**Azure AI Content Safety groundedness detection** (AWS equivalent: Guardrails contextual grounding check).
Q4	Schedule the KB **StartIngestionJob** (EventBridge Scheduler, serverless) after the nightly data load to S3.
Q5	Retrieval ranking problem → add a **reranker** (Bedrock rerank model / Azure semantic ranker).

Next: Day 2 — Agents and tool calling (Wed 14 Oct)
Agent loop (reason → act → observe), Bedrock Agents, action groups (Lambda + OpenAPI schema / function details), return of control, attaching a Knowledge Base, session memory. Prep: basic Python Lambda handler.